

Major Testing Activities in Generic Testing Process

Course Code: CSC4133

Course Title: Software Quality and Testing



Dept. of Computer Science
Faculty of Science and Technology

Lecturer No:	10	Week No:	5	Semester:	
Lecturer:	<i>Prof. Dr. kamruddin Nur, kamruddin@aiub.edu</i>				

Lecture Outline



- Major Testing Activities in Generic Testing Process
 - 1) Test planning and preparation
 - 2) Test Execution
 - 3) Test Analysis and follow-up activities

Objectives and Outcomes



- **Objectives:** To understand the major testing activities in the generic testing process.
- **Outcomes:** Students are expected to be able to explain the major testing activities in the generic testing process – test planning and preparation, test execution, analysis and follow-up activities.

Generic Testing Process



- Major Testing Activities in the Generic Testing Process:

- 1) Test planning and preparation

- Goal setting
- Information gathering
- Model construction
- Test cases
- Test procedures

- 2) Test Execution

- Observation & measurements of product behavior

- 3) Test Analysis & Follow-up activities

- Result checking and analysis (to determine if a failure has been observed)
- Follow-up activities



[1] Test Planning and Preparation

- Most important activity in the generic testing process for systematic testing based on formal models.
- Most of the key decisions about testing are made during this stage.
- Test planning & test preparation are sometimes treated as separate groups of activities.
 - **High-level** test planning
 - **Low-level** activities related to test preparation

[1] Test Planning and Preparation



- **Test Planning:** The high-level task for test planning is to set goals & to determine a general testing strategy.
 - Goal setting
 - Overall strategy
- **Test Preparation:**
 - Preparing test cases & test suite(s)
 - Preparing test procedure

Test Planning



- **Goal setting**
 - Quality perspectives of the customer
 - Quality expectations of the customer
 - Mapping to internal goals and concrete (quantified) measurement
 - e.g. customer's correctness concerns ==> Specific reliability target
- **Overall strategy**, including:
 - Specific objects to be tested
 - Techniques to use
 - Measurement data to be collected
 - Analysis and follow-up activities
- **Key:** plan the *“whole thing”*!

Test Planning



- We set an overall testing strategy by making the following decisions:
- Overall objectives and goals, which can be refined into specific goals for specific testing. Some specific goals include reliability for usage-based statistical testing or coverage for various traditional testing techniques.
- Objects to be tested and the specific focus: **Functional testing** views the software product as a *black-box* and focuses on testing the external functional behavior; while **structural testing** views the software product or component as *white-box* and focuses on testing the internal implementation details.

Test Planning



- Once the overall testing strategy has been selected, we can plan to allocate resources and staff to implement it. The available staff & resources also affect the specific models and techniques that can be used to implement the strategy.
- **Sometimes, existing test suites can be used** with some minor modifications or adaptations, which would require minimal additional effort in test planning & preparation.

Test Preparation



- Procedure for test preparation
 - Preparing test cases
 - Individual test cases
 - Test case allocation
 - Preparing test procedure
 - Basis for test procedure
 - Order, flow, follow-up

Test Preparation



■ General concepts:

- Input variable: test point
- Input space: all possible input variable values
- Test case: *A **test case** is a **document** that describes an input, action, or event, and its expected results, in order to determine if a feature of an application is working correctly.*

Test case = **static object** + **input** to enable test to

start → execute → finish

- Test run: A test run is a **dynamic** unit of specific test activities in the overall testing sequence on a selected testing object.

Test run ==> operation instances

Individual Test Case Preparation



- Individual test cases (micro-level) vs. test suite (macro-level)
- From multiple sources:
 - Actual runs (usage-based)
 - Implementation-based (white-box)
 - Specification-based (black-box)
 - May use similar/earlier products
 - (direct) “record-and-replay”



Test cases based on Formal Models

- Most organized, systematic test cases are derived from formal testing models:
 - Directly via newly constructed models
 - Indirectly via existing test cases, etc.

Test Suite Preparation



■ Test Suite (macro-level) :

- The collection of individual test cases that will be run in a test sequence until some stopping criteria are satisfied is called a *test suite*.
 - Test suite preparation involves the construction & allocation of individual test cases in some systematic way based on the specific testing techniques used.
 - Another way to obtain a test suite is through **reuse of test cases for earlier versions of the same product**. This kind of testing is commonly referred to as *regression testing*.
 - In general, all the test cases should form an integrated suite, regardless of their origin, how they are derived, and what models are used to derive them.

Test Suite Preparation



- There are many ways to organize the test suite or test suites. The most common way is to organize them by sub-phases, because of the different objects, objectives, concerns, perspectives, priorities, and the testing techniques used.
- Various attributes can be used to describe, classify, and organize individual test cases in the suite.

Test Suite Preparation



- **Test Suite**
 - ▶ **Existing suite:** What & Where? Suitability? Selection/Screening?
 - ▶ **Construction** /generation of **new ones**
 - ▶ Organization & management: Often hierarchical
- **Adding new test cases**
 - ▶ Estimate # of new test cases
 - ▶ Specify new(individual) test cases
 - ▶ Integrate to existing test cases
- **Allocation to systems/operations**
 - ▶ OP-based/structured-based allocation
 - ▶ Both old & new test cases in suite



Test Procedure Preparation

- In addition to preparation of individual test cases and the overall test suite, the *test procedure* also needs to be prepared for effective testing.
- The basic question is the sequencing of the individual test cases and the switch-over from one test run to another.
- KEY consideration: Sequencing of the individual test cases
 - General: simple to complex
- Several concerns affect the specific test procedure to be used –
 - *Dependencies* among individual test cases
 - *Defect detection* related sequencing
 - *Problem diagnosis* related sequencing

Test Procedure Preparation



- **Other considerations:** A related topic to test procedure preparation is the assignment of people to perform certain tests. Their roles & responsibilities
 - Effectiveness/efficiency concerns
 - Smooth transition between test runs
 - Management/resource/personnel/etc.

[2] Test Execution



[2] Test Execution

- The key to the overall test execution is the smooth transition from one test run to another, which also requires us to allocate all the required resources to ensure that individual test runs can be started, executed, and finished, and related problems can be handled seamlessly.
 - ▶ Execution planning and management
 - ▶ Related activities: important part
 - Failure identification and measurement
 - Other measurement

[2] Test Execution



- General steps in Test Execution:
 - a) Allocating test time & resources
 - b) Invoking and running tests, and collecting execution information & measurements
 - c) Checking testing results & identifying system failures

[2] Test Execution



■ Allocating Test Time:

- Test time & resources allocation is most closely related to the test planning & preparation activities . Although the allocation could be planned or even carried out at the previous stage, the monitoring, adjustment, and management of these resources need to be carried out during test execution.
- ▶ OP-based: systems/features/operations
- ▶ Coverage-based: func./struc. areas
- ▶ Alternatives : bottom-up approach
 - Individual test cases ==> test time
 - Sum-up ==> overall allocation

[2] Test Execution



- Invoking test : OP-based/coverage-based
- Common part : Retest due to defect-fix
 - ▶ Defect fix ==> verify fix
 - ▶ Code-base or feature change
 - ▶ General *regression test*

[2] Test Execution



- Identifying system failures (oracle problem):
- Test oracle: Any means to check the testing result
 - ▶ Similar for OP-based/coverage-based
 - ▶ Analyze test output for deviations
 - ▶ Determine: deviation=failure?
 - ▶ Handling normal vs. failed runs

[2] Test Execution



- **Failure observation and measurement:**
 - ▶ Determine: deviation = failure ?
 - ▶ Establish when failure occurred
 - ▶ Collect failure information: What /where/when/severity/etc.
- **Defect handling and test measurement:**
 - ▶ Defect status & change (controlled)
 - ▶ Information gathering during testing
 - ▶ Follow-up activities:
 - Fix-verification cycle
 - Other possibilities (defer, invalid, etc.)

Test Execution Measurement



- Observed failures need to be recorded & tracked until their resolution.
- Detailed information about failure observations & related activities is needed for problem diagnosis and defect fixing.
- Some specific information for failures includes various generic information about defects – defect type, severity, impact areas, possible cause, when-injected, etc. This information could be collected when failure observed & recorded or when faults were fixed, or even afterward.
- Successful executions also need to be recorded for various purposes (documentation of testing activities, to check future execution results)
- Organizations use template for test execution measurements.



[3] Test Analysis and Follow-up Activities

[3] Test analysis and follow-up:

- The third group of major testing activities is analysis and follow-up after test execution. The measurement data collected during test execution, together with other data about the testing & overall environment, form the data input to these analyses which provide valuable feedback to test execution and other testing & development activities. Direct follow-up includes defect fixing and making other management decisions, such as product release and transition from one development phase/sub-phase to another.
 - ▶ Execution/other measurement analyzed
 - ▶ Analysis results as basis for follow-up
 - ▶ Feedback & Follow-up:
 - Decision making (exit testing? etc.)
 - Adjustment and improvement

[3] Test Analysis and Follow-up Activities



- **Input to analysis**
 - ▶ Test execution information
 - ▶ Particularly failure cases
 - ▶ Timing & characteristics data
- **Analysis and output**
 - ▶ Basic individual (failure) case
 - Problem identification/reporting
 - Repeatable problem setup
 - ▶ Overall reliability and other analysis?
- **Follow-up activities:**
 - ▶ Defect analysis & removal (& **re-test**)
 - ▶ Decision making & management
 - ▶ Test process & quality improvement



[3] Test Analysis and Follow-up Activities

- Analysis & Follow-up for **individual test runs**:
 - ▶ Success: continue with normal testing
 - ▶ Failure: *see below*
- Analysis and follow-up for **failed runs**:
 - Understanding the problem by studying the execution record
 - Recreating the problem (confirmation)
 - Problem diagnosis (to examine what kind of problem it is, where, when, and possible causes) ==> may involve multiple related runs
 - Locating the faults
 - Defect fixing (fault removal)
 - ▶ Commonly via add/remove/modify code
 - ▶ Sometimes involve design & requirement changes; Re-run/re-test to confirm defect fixing

[3] Test Analysis and Follow-up Activities



- Analysis & Follow-up for **overall testing results:**
 - ▶ Reliability analysis and follow-up
 - ▶ Coverage analysis and follow-up
 - ▶ Defect analysis and follow-up
- Analyses: Different focuses
 - Overall reliability and coverage for usage-based & coverage-based testing
 - Detailed defect analysis
- Follow-up activities:
 - Decision making & management
 - Test process & quality improvement



Books

- *Software Quality Engineering: Testing, Quality Assurance and Quantifiable Improvement*, by Jeff Tian



References

1. *Software Testing and Quality Assurance: Theory and Practice*, by Kshirasagar Naik, Priyadarshi Tripathy
2. *Software Quality Assurance: From Theory to Implementation*, by Daniel Galin
3. *Software Testing and Continuous Quality Improvement*, by William E. Lewis
4. *The Art of Software Testing*, by Glenford J. Myers, Corey Sandler and Tom Badgett
5. *Software Testing Fundamentals: Methods and Metrics* by Marnie L. Hutcheson